

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN
VIỆN TRÍ TUỆ NHÂN TẠO



BÁO CÁO BÀI TẬP

PHÁT HIỆN, ĐỊNH VỊ VÀ DECODE MÃ QR
BẰNG XỬ LÝ ẢNH

Môn học: 2526I_AIT3001* - Image Processing and Analysis

Giảng viên hướng dẫn: TS. Trần Quốc Long

Người thực hiện: Hoàng Minh Quyền 23020421

Hà Nội, 2026

Mục lục

1	Giới Thiệu	3
2	Mô Tả Phương Pháp	3
2.1	Tiền xử lý ảnh	4
2.1.1	Giới hạn kích thước và cache	4
2.1.2	Chuyển ảnh xám và Gaussian Blur	4
2.1.3	Nhị phân hóa	5
2.1.4	Morphological Close	5
2.1.5	Chiến lược đa cấu hình	5
2.2	Phát hiện vùng chứa mã QR	6
2.2.1	Phát hiện Finder Pattern qua cây đường viền	6
2.2.2	Phân nhóm Triplet và nội suy góc thứ tư	7
2.2.3	Điểm heuristic và phân loại Random Forest	8
2.2.4	Non-Maximum Suppression	10
2.3	Giải mã nội dung mã QR	11
2.3.1	Bước 1: Perspective Warp và nhị phân hóa	11
2.3.2	Bước 2: Lấy mẫu lưới căn chỉnh theo Finder Pattern	11
2.3.3	Bước 3: Đọc Format Information	12
2.3.4	Bước 4: Trích xuất và giải mã dữ liệu	12
2.3.5	Bước 5: Reed-Solomon Error Correction	12
2.3.6	Bước 6: Phân tích payload	13
3	Các Kỹ Thuật Sử Dụng	13
3.1	Thư viện và công cụ	14
3.2	Mô hình và thuật toán chính	15
3.2.1	Contour Hierarchy — RETR_TREE	15
3.2.2	Perspective Transform — Homography	15
3.2.3	Random Forest Classifier	15
3.2.4	Reed-Solomon trên $GF(2^8)$	21
3.2.5	Non-Maximum Suppression với Convex Overlap Ratio	22
3.3	Tối ưu hóa hiệu năng	22
3.3.1	Kiểm soát thread — tránh thread contention	22
3.3.2	Multiprocessing với ProcessPoolExecutor	23
3.3.3	Cache ảnh đã resize	24
3.3.4	Chế độ tùy chọn giải mã	24
3.3.5	Chế độ debug visual	25

4	Kết Quả Trên Tập Public	25
4.1	Tổng quan tập dữ liệu public	25
4.2	Tiêu chí đánh giá	25
4.3	Độ chính xác phát hiện	26
4.4	Tốc độ xử lý	26
4.5	Trường hợp đúng / sai tiêu biểu	27
5	Phân Tích và Thảo Luận	27
5.1	Quá trình phát triển và các bước ngoặt kỹ thuật	27
5.2	Khó khăn gặp phải	28
5.3	Hạn chế của phương pháp	29
5.4	Hướng cải thiện	30
6	Kết Luận	31
A	Hướng dẫn Chạy Chương trình	32
A.1	Cài đặt môi trường	32
A.2	Chạy phát hiện và giải mã	32

1 Giới Thiệu

Mã QR (Quick Response Code) là tiêu chuẩn mã vạch hai chiều phổ biến nhất hiện nay, được chuẩn hóa theo ISO/IEC 18004, ứng dụng rộng rãi trong thanh toán điện tử, quản lý chuỗi cung ứng và tiếp thị kỹ thuật số. Nhiệm vụ của bài tập lớn là xây dựng hệ thống *tự động phát hiện, định vị và giải mã* mã QR từ ảnh thực tế **không** dùng thư viện giải mã sẵn có (ZBar, ZXing, cv2.QRCodeDetector), mà triển khai từ đầu bằng kỹ thuật xử lý ảnh thuần túy.

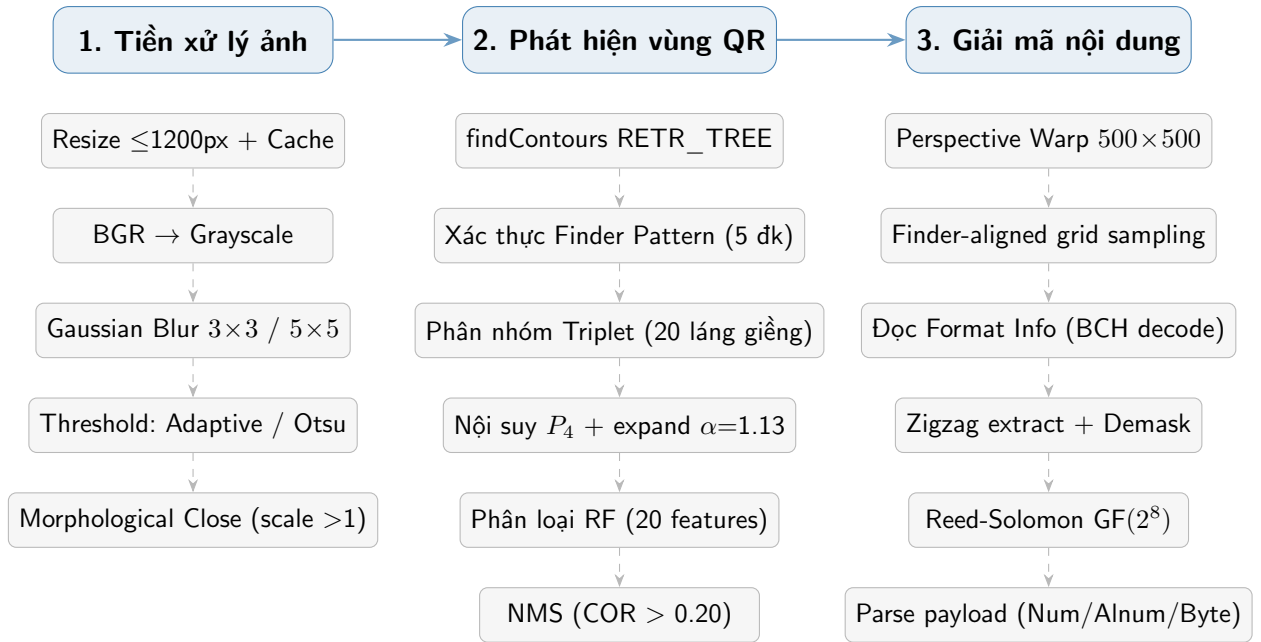
Các thách thức chính gặp phải: chiếu sáng không đồng đều, nhiễu quang học, biến dạng phối cảnh, mã QR bị xoay hoặc có kích thước nhỏ, ảnh chứa đồng thời nhiều mã QR. Để đối phó, nhóm kết hợp pipeline xử lý ảnh đa tầng với bộ phân loại Random Forest được huấn luyện từ dữ liệu ground truth.

Kết quả được đánh giá **chỉ trên tập public** theo tiêu chí F1 Score (trọng số 0,7) và Speed_Score (trọng số 0,3). Tập private do giảng viên chạy độc lập và không được dùng để tinh chỉnh hay báo cáo.

Báo cáo được tổ chức theo 4 mục: Mô tả phương pháp (Mục 2), Các kỹ thuật sử dụng (Mục 3), Kết quả trên tập public (Mục 4), Phân tích và thảo luận (Mục 5).

2 Mô Tả Phương Pháp

Hệ thống được tổ chức thành ba giai đoạn nối tiếp (Hình 1): **(1)** Tiền xử lý ảnh, **(2)** Phát hiện vùng chứa QR, và **(3)** Giải mã nội dung. Với mỗi ảnh đầu vào, pipeline chạy tuần tự qua 5 cấu hình scale/mode khác nhau; kết quả từ tất cả cấu hình được gộp lại và lọc trùng lặp ở giai đoạn cuối, đảm bảo phát hiện được mã QR ở nhiều kích thước và điều kiện chiếu sáng khác nhau.



Hình 1: Sơ đồ pipeline ba giai đoạn chi tiết của hệ thống phát hiện mã QR

2.1 Tiền xử lý ảnh

2.1.1 Giới hạn kích thước và cache

Hàm `resize_for_inference(image, max_side=1200)` kiểm tra cạnh dài của ảnh. Nếu vượt quá 1200 px, ảnh được thu nhỏ bằng phép nội suy `cv2.INTER_AREA` (giảm tốt nhất cho downscale):

```

1 def resize_for_inference(image, max_side=1200):
2     h, w = image.shape[:2]
3     scale = 1.0
4     if max(h, w) > max_side:
5         scale = max_side / float(max(h, w))
6         image = cv2.resize(image, (int(w*scale), int(h*scale)),
7                             interpolation=cv2.INTER_AREA)
8     return image, scale
  
```

Tỷ lệ `base_scale` được lưu lại để quy đổi tọa độ phát hiện về hệ tọa độ ảnh gốc ở cuối pipeline. Trong vòng lặp 5 cấu hình, mỗi phiên bản ảnh ở cùng scale chỉ được tính toán **một lần** nhờ cơ chế cache `cached_img[scale]`, tránh resize lặp lại.

2.1.2 Chuyển ảnh xám và Gaussian Blur

Ảnh BGR được chuyển sang grayscale bằng `cv2.cvtColor(..., COLOR_BGR2GRAY)`, giảm 2/3 dữ liệu trong khi giữ nguyên thông tin cấu trúc không gian. Sau đó áp dụng bộ lọc Gaussian để giảm nhiễu hạt trước bước nhị phân hóa:

- **Chế độ normal:** Gaussian Blur 3×3 , kernel nhỏ để tránh làm nhòe cấu trúc module QR.
- **Chế độ otsu:** Gaussian Blur 5×5 , làm mịn mạnh hơn vì Otsu hoạt động tốt nhất với histogram đã được làm trơn.

2.1.3 Nhị phân hóa

Hệ thống sử dụng hai chiến lược nhị phân hóa tùy theo chế độ cấu hình:

Adaptive Thresholding (chế độ normal). `cv2.adaptiveThreshold` với `ADAPTIVE_THRESH_GAUSSIAN_C` và `THRESH_BINARY_INV`. Block size được tính thích nghi theo scale:

$$B = \max(11, \min(\lfloor 41 \cdot \text{scale} \rfloor, 51)) \quad (1)$$

trong đó toán tử $\lfloor \cdot \rfloor$ đảm bảo block size luôn lẻ (yêu cầu của OpenCV). Hằng số $C = 2$ được trừ khỏi trung bình cục bộ. Chiến lược này hiệu quả với điều kiện chiếu sáng không đồng đều vì tính ngưỡng riêng cho từng vùng cục bộ của ảnh.

Otsu's Thresholding (chế độ otsu). Tìm ngưỡng toàn cục t^* tối đa hóa phương sai giữa hai lớp pixel [2]. Flag `THRESH_BINARY_INV` được dùng để đảm bảo Finder Pattern (màu đen) trở thành vùng sáng trên ảnh nhị phân, phù hợp với hàm `findContours`:

$$\sigma_B^2(t) = \omega_0(t) \omega_1(t) [\mu_0(t) - \mu_1(t)]^2 \quad (2)$$

2.1.4 Morphological Close

Với $\text{scale} > 1.0$, thêm bước Morphological Closing (kernel hình chữ nhật 2×2) để hàn gắn các khoảng đứt nhỏ giữa các module QR phát sinh do chất lượng in ấn hoặc nhiễu ảnh khi phóng to:

```
1 if scale > 1.0:
2     kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
3     thresh = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```

2.1.5 Chiến lược đa cấu hình

Hệ thống chạy tuần tự **5 cấu hình** (Bảng 1). Mỗi cấu hình nhắm vào một nhóm mã QR cụ thể: scale nhỏ ($0.5 \times$) phù hợp với QR lớn chiếm nhiều diện tích ảnh; scale lớn ($2.5 \times$) phóng to để phát hiện QR nhỏ vốn có Finder Pattern dưới ngưỡng diện tích tối thiểu ở scale gốc.

Bảng 1: 5 cấu hình tiền xử lý chạy tuần tự trong CONFIGS

STT	Scale	Mode	Mục đích
1	1.0	normal	Phát hiện chuẩn; xử lý chiếu sáng không đồng đều
2	0.5	normal	QR lớn; downscale giảm nhiễu, tăng tốc
3	0.5	otsu	QR lớn; ánh sáng đồng đều, histogram bimodal
4	1.5	normal	QR vừa; tăng phân giải để cải thiện phát hiện
5	2.5	normal	QR nhỏ; phóng to đủ để Finder Pattern vượt ngưỡng

2.2 Phát hiện vùng chứa mã QR

2.2.1 Phát hiện Finder Pattern qua cây đường viền

Finder Pattern là ba hình vuông lồng nhau (đen ngoài – trắng giữa – đen lõi) đặt ở ba góc mỗi mã QR theo chuẩn ISO/IEC 18004, tạo ra cấu trúc phân cấp đường viền 3 tầng rất đặc trưng. Hàm `cv2.findContours` với chế độ `RETR_TREE` và `CHAIN_APPROX_SIMPLE` tái tạo toàn bộ cây phân cấp này. Mỗi đường viền được đưa vào hàm `_is_valid_finder_pattern` để kiểm tra 5 điều kiện cứng:

1. **Tỷ lệ khung hình (Aspect Ratio):** $AR = w/h \in [0.4, 2.5]$. Finder Pattern là hình vuông, cho phép dung sai để bù đắp biến dạng phối cảnh nhẹ và pixel rounding.
2. **Diện tích vòng ngoài:** $area_{outer} \geq 30 \cdot scale^2$ px. Ngưỡng nhân với $scale^2$ vì diện tích tỷ lệ với bình phương hệ số zoom.
3. **Kiểm tra cấu trúc lồng 3 tầng:**
 - Đường viền con (vòng trắng, `hier[idx][2]`) phải tồn tại với diện tích $\geq 8 \cdot scale^2$ px.
 - Đường viền cháu (lõi đen, `hier[child][2]`) phải tồn tại với diện tích $\geq 2 \cdot scale^2$ px.
4. **Tỷ lệ diện tích giữa 3 tầng:**

$$\frac{area_{outer}}{area_{middle}} \in (1.2, 4.5) \quad (3)$$

$$\frac{area_{outer}}{area_{inner}} \in (3.0, 16.0) \quad (4)$$

Các khoảng này được xác định thực nghiệm để bao phủ các phiên bản QR từ nhỏ đến lớn (biến dạng tỷ lệ theo căn bậc hai của diện tích).

5. **Tính đồng tâm:** Khoảng cách Euclidean giữa tâm vòng ngoài và tâm lõi trong (tính từ moments bậc 0 và bậc 1):

$$\left\| \begin{pmatrix} c_{x,\text{out}} \\ c_{y,\text{out}} \end{pmatrix} - \begin{pmatrix} c_{x,\text{in}} \\ c_{y,\text{in}} \end{pmatrix} \right\|_2 < 0.2 \cdot \bar{w}_{\text{fp}} \quad (5)$$

trong đó $\bar{w}_{\text{fp}} = (w + h)/2$ là chiều rộng trung bình của Finder Pattern.

Các điểm tâm ứng viên cách nhau dưới 8 px được gộp lại (chỉ giữ điểm đầu tiên), tránh báo trùng cùng một Finder Pattern do nhiều đường viền kề nhau.

2.2.2 Phân nhóm Triplet và nội suy góc thứ tư

Sau khi có danh sách Finder Pattern, hàm `group_patterns_to_candidates` duyệt mọi tổ hợp bộ ba trong **20 láng giềng gần nhất** của mỗi điểm (giảm độ phức tạp từ $O(n^3)$ xuống $O(n \cdot k^2/2)$ với $k = 20$).

Mỗi bộ ba (P_A, P_B, P_C) phải vượt qua các ràng buộc heuristic:

a) Đồng kích thước.

$$\text{size_var} = \frac{\max(w_A, w_B, w_C)}{\min(w_A, w_B, w_C)} \leq 2.2 \quad (6)$$

Ba Finder Pattern của cùng một mã QR phải có kích thước xấp xỉ nhau.

b) Khoảng cách hợp lý.

$$d_{\text{max}}/\bar{w} \leq 35.0 \quad (7)$$

$$d_{\text{min}}/\bar{w} \geq 1.2 \quad (8)$$

trong đó $\bar{w} = (w_A + w_B + w_C)/3$. Điều kiện này loại bỏ các bộ ba quá rộng (không phải cùng một mã) hoặc quá hẹp (ba điểm quá gần nhau).

c) Tam giác gần vuông. Cạnh dài nhất trong ba cạnh (d_{AB}, d_{AC}, d_{BC}) được chọn là cạnh huyền, hai điểm đầu kia là hai cạnh góc vuông. Góc đỉnh phải thỏa:

$$r = \frac{\min(d_1, d_2)}{\max(d_1, d_2)} \geq 0.35 \quad \text{và} \quad |\cos \theta| \leq 0.75 \quad (9)$$

d) Kiểm tra độ kết cấu (Quick Texture Gate). Bộ ba vượt qua các điều kiện hình học được warp thành ảnh 96×96 px (với $\alpha_{\text{detect}} = 0.68$ để bao vùng nhỏ hơn).

Mật độ cạnh Canny vùng trung tâm [20:76, 20:76] phải đạt:

$$\text{ed_raw} = \frac{\#\{\text{pixel cạnh}\}}{56^2} \geq 0.12 \quad (10)$$

Ngưỡng này loại nhanh các bộ ba hình học hợp lệ nhưng nằm trên vùng ảnh trống (nền đồng màu, không có module QR).

e) Không có Finder Pattern xâm nhập. Kiểm tra xem có Finder Pattern nào khác (ngoài bộ ba hiện tại) nằm bên trong polygon phát hiện không:

```

1 intruder_count = sum(
2     1 for p in patterns
3     if p not in (pA, pB, pC)
4     and point_in_polygon(p, detect_poly)
5 )
6 if intruder_count > 0: continue

```

Điều kiện này đảm bảo không ghép nhầm các Finder Pattern của hai mã QR khác nhau trong ảnh chứa nhiều mã.

Nội suy góc thứ tư P_4 . Góc dưới-phải của mã QR không có Finder Pattern, được ước tính bằng phép cộng vector:

$$P_4 = P_{\text{TR}} + P_{\text{BL}} - P_{\text{TL}} \quad (11)$$

Polygon kết quả được mở rộng với hai hệ số alpha khác nhau:

- $\alpha = 0.68$: polygon nhỏ hơn dùng để kiểm tra texture và đếm intruder.
- $\alpha = 1.13$: polygon lớn hơn dùng làm bounding box đầu ra, bao phủ đủ vùng margin quiet zone của mã QR.

2.2.3 Điểm heuristic và phân loại Random Forest

Điểm heuristic hệ thống (sys_score). Mỗi bộ ba vượt qua tất cả điều kiện được tính điểm:

$$s_{\text{sys}} = \frac{d_{AB} + d_{AC} + d_{BC}}{3 \cdot d_{\text{diag}}} - \text{ed_raw} + |1 - r| \quad (12)$$

Điểm thấp = tốt hơn: bộ ba có trọng tâm nhỏ hơn, kết cấu phong phú hơn, và cân bằng hơn được ưu tiên.

Phân loại bằng Random Forest. Khi model RF có sẵn, 20 đặc trưng (Bảng 2) được tính và gộp thành batch, sau đó `RF.predict_proba` chạy một lần duy nhất cho tất cả ứng viên trong ảnh (tránh overhead gọi nhiều lần):

```

1 X = np.array(pending_features, dtype=np.float32)
2 probs = RF_MODEL.predict_proba(X)[: , 1]
3 for aabb, prob, sys_s in zip(pending_aabbs, probs,
4     pending_scores):
5     if prob >= 0.5:
6         candidates.append({
7             'score': (1.0 - prob) + sys_s * 0.001,
8             'poly': aabb
9         })

```

Điểm tổng hợp ưu tiên xác suất RF ($1 - \hat{p}$, giá trị thấp = tốt) và dùng s_{sys} như một tiebreaker với trọng số nhỏ. Khi chưa có model, hệ thống rơi vào chế độ **Heuristic Fallback**: dùng trực tiếp s_{sys} làm điểm sắp xếp.

20 đặc trưng đầu vào. Bảng 2 liệt kê đầy đủ 20 đặc trưng. Đáng chú ý là ba đặc trưng cấu trúc QR được tính từ patch 96×96 đã warp:

- **corner_top2/3**: mỗi góc của patch được cắt ra ($s = \lfloor 96 \times 0.28 \rfloor = 26$ px), nhị phân hóa, và chạy hàm **run_length_ratio_score** để đo khớp với tỷ lệ 1:1:3:1:1 — signature của Finder Pattern theo ISO.
- **timing_score**: đo tính giao thời B-W-B-W dọc theo 6 dòng quét qua patch (vị trí 22%, 50%, 78% theo cả hàng và cột).
- **module_score**: ảnh nhị phân 96×96 được pooling về 24×24 và đo tần suất chuyển đổi ngang/dọc, phản ánh tính phân ô đều của lưới module QR.

Bảng 2: 20 đặc trưng đầu vào cho mô hình Random Forest

Nhóm	Tên	Ý nghĩa và cách tính
Hình học Triplet (5)	ratio	$\min(d_1, d_2) / \max(d_1, d_2)$: tỷ lệ 2 cạnh góc vuông
	cos_val	$ \cos \theta $ tại đỉnh tam giác
	proximity	$(d_{AB} + d_{AC} + d_{BC}) / (3 d_{\text{diag}})$
	size_variance	$\max(w_i) / \min(w_i)$: độ lệch kích thước 3 FP
	edge_density_raw	Mật độ cạnh Canny vùng $[20:76]^2$ của patch
Kết cấu ảnh Patch 96^2 (6)	black_ratio	$\bar{1}[\text{bw} = 0]$: tỷ lệ pixel đen
	edge_density	Mật độ cạnh Canny toàn patch
	balance_score	$1 - \min(\text{black_ratio} - 0.5 / 0.5, 1)$
	center_mean	$\bar{g}_{[24:72, 24:72]} / 255$: TB vùng tâm
	center_std	$\text{std}(g_{[24:72, 24:72]}) / 255$
	poly_area_norm	$\text{Area}(\text{polygon}) / (\text{Area}_{\text{img}} \cdot \text{scale}^2)$
Cấu trúc QR (4)	corner_top2	TB score 1:1:3:1:1 tại 2 góc tốt nhất
	corner_top3	TB score 1:1:3:1:1 tại 3 góc tốt nhất
	timing_score	Score B-W-B-W qua 6 dòng quét
	module_score	$0.5(\bar{h}_{\text{flip}} + \bar{v}_{\text{flip}})$ trên pooled 24^2
Hình học bổ sung (5)	avg_fp_width	$\bar{w}_{\text{fp}} / \max(H, W)$: kích thước FP chuẩn hóa
	intruder_count	Số FP khác xâm nhập vào polygon (=0 sau lọc)
	dist_to_size	$d_{\text{max}} / \bar{w}_{\text{fp}}$
	hyp_ratio	$d_{\text{hyp}} / (d_1 + d_2)$: tỷ lệ cạnh huyền
	aspect_diag	$\min(\text{diag}_1, \text{diag}_2) / \max(\text{diag}_1, \text{diag}_2)$

2.2.4 Non-Maximum Suppression

Tất cả ứng viên từ 5 cấu hình được gộp vào một danh sách, sắp xếp theo điểm số tăng dần (ứng viên tốt nhất đứng đầu). NMS giữ lại box tiếp theo khi Convex Overlap Ratio (COR) với mọi box đã chọn không vượt quá 0.20:

$$\text{COR}(P, Q) = \frac{\text{Area}(P \cap Q)}{\min(\text{Area}(P), \text{Area}(Q))} \quad (13)$$

COR được dùng thay cho IoU thông thường vì nó không phạt khi một box lớn chứa trọn một box nhỏ — trường hợp phổ biến khi cùng một mã QR được phát hiện ở hai scale khác nhau với kích thước polygon khác nhau. Tọa độ các góc của box cuối cùng được clip về $[0, W - 1] \times [0, H - 1]$ trước khi ghi ra CSV.

2.3 Giải mã nội dung mã QR

Hàm `decode_qr_content(image, poly)` nhận polygon 4 góc đã phát hiện và thực hiện quy trình giải mã đầy đủ **từ đầu**, không dùng thư viện QR bên ngoài. Toàn bộ bộ giải mã — từ GF(2⁸) arithmetic đến Reed-Solomon đến parser payload — được triển khai thủ công trong `main.py`.

2.3.1 Bước 1: Perspective Warp và nhị phân hóa

Polygon được warp về ảnh vuông 500×500 px bằng `cv2.getPerspectiveTransform` + `cv2.warpPerspective`, loại bỏ biến dạng phối cảnh. Ba chiến lược nhị phân hóa được thử tuần tự để tối đa hóa tỷ lệ thành công:

1. Gaussian Blur 3×3 + Otsu
2. Gaussian Blur 5×5 + Adaptive (block=31, $C = 2$)
3. Ngưỡng cố định = $\lfloor \text{median}(I) \times 0.85 \rfloor$

2.3.2 Bước 2: Lấy mẫu lưới căn chỉnh theo Finder Pattern

Thay vì giả định ảnh đã thẳng, hàm `_decode_finder_aligned` tái phát hiện Finder Pattern trực tiếp trên ảnh warped, tính vector hướng từ đỉnh TL đến TR ($\vec{d}_c$) và từ TL đến BL ($\vec{d}_r$), sau đó lấy mẫu lưới theo hướng vector thực tế:

```
1 px = origin[0] + (c + 0.5)*ms*dir_c[0] + (r + 0.5)*ms*dir_r[0]
2 py = origin[1] + (c + 0.5)*ms*dir_c[1] + (r + 0.5)*ms*dir_r[1]
```

Điểm gốc (origin) được xác định từ tâm Finder Pattern TL lùi về $3.5 \times ms$ theo cả hai hướng. Hệ thống thử tất cả 20 phiên bản QR ($v = 1 \dots 20$) và 2 hướng hoán vị của vector ($\vec{d}_c, \vec{d}_r$) để xử lý trường hợp mã QR bị xoay. Mỗi ô module được lấy mẫu tại tâm và 8 điểm lân cận (offset $\pm \lfloor ms \times 0.15 \rfloor$), quyết định đen khi $\geq 5/9$ điểm là đen.

Nếu không tìm được đủ Finder Pattern trên ảnh warped, fallback về hàm `_try_decode_boundary`: quét biên pixel để xác định vùng QR, ước lượng phiên bản qua Timing Pattern, thử 4 hướng xoay $\times$ nhiều offset biên.

2.3.3 Bước 3: Đọc Format Information

Hàm `_read_format_info` đọc 15-bit format information từ hai vị trí cố định quanh Finder Pattern TL (và góc TR, BL). So khớp với bảng 32 mã hợp lệ theo thuật toán BCH bằng khoảng cách Hamming tối thiểu (chấp nhận tối đa 3 bit lỗi):

```
1 for d in range(32):
2     dist = bin(bits_15 ^ _FORMAT_INFO_TABLE[d]).count('1')
3     if dist < min_d:
4         min_d = dist; best = d
5 if min_d > 3: return None, None
```

Output: cấp độ sửa lỗi (L/M/Q/H) và chỉ số mặt nạ (0–7).

2.3.4 Bước 4: Trích xuất và giải mặt nạ dữ liệu

Hàm `_extract_data_bits` duyệt ảnh theo zigzag từ góc dưới-phải lên trên-trái (bỏ qua các ô thuộc Function Pattern như Finder Pattern, Timing Pattern, Alignment Pattern, Format Information). Với mỗi ô dữ liệu, giá trị được XOR với hàm mặt nạ tương ứng — một trong 8 hàm chuẩn:

Mặt nạ	Điều kiện bật (r, c)
0	$(r + c) \bmod 2 = 0$
1	$r \bmod 2 = 0$
2	$c \bmod 3 = 0$
3	$(r + c) \bmod 3 = 0$
4	$(\lfloor r/2 \rfloor + \lfloor c/3 \rfloor) \bmod 2 = 0$
5	$(rc) \bmod 2 + (rc) \bmod 3 = 0$
6	$((rc) \bmod 2 + (rc) \bmod 3) \bmod 2 = 0$
7	$((r + c) \bmod 2 + (rc) \bmod 3) \bmod 2 = 0$

2.3.5 Bước 5: Reed-Solomon Error Correction

Bộ sửa lỗi RS được triển khai hoàn toàn thủ công trên trường Galois $GF(2^8)$ với đa thức nguyên thủy $p(x) = x^8 + x^4 + x^3 + x^2 + 1$ (hệ số 0x11D). Bảng logarithm và antilog được khởi tạo một lần duy nhất khi load module. Quy trình sửa lỗi cho mỗi block dữ liệu:

$$s_i = \sum_{j=0}^{n-1} c_j \cdot \alpha^{ij}, \quad i = 1, \dots, 2t \quad (14)$$

Tính syndrome ($s_0, \dots, s_{2t-1}$)

Hàm `_rs_calc_syndromes`: đánh giá đa thức codeword tại các điểm $\alpha^0, \dots, \alpha^{2t-1}$.

Nếu tất cả syndrome bằng 0: không có lỗi, trả về ngay.

Berlekamp-Massey — tìm đa thức định vị lỗi $\Lambda(x)$

Hàm `_rs_berlekamp_massey`: thuật toán lặp xây dựng đa thức $\Lambda(x)$ có bậc bằng số lỗi e . Điều kiện có thể sửa: $2e \leq 2t$.

Chien Search — tìm vị trí lỗi

Hàm `_rs_chien_search`: tìm tất cả nghiệm của $\Lambda(x)$ bằng cách thử tất cả n phần tử của trường. Vị trí lỗi là nghịch đảo của nghiệm tìm được.

Forney Algorithm — tính giá trị sửa lỗi

Hàm `_rs_forney`: tính đa thức đánh giá lỗi $\Omega(x) = s(x) \cdot \Lambda(x) \bmod x^{2t}$, rồi áp dụng công thức Forney để tính giá trị sai khác tại mỗi vị trí lỗi:

$$e_i = -X_i \cdot \frac{\Omega(X_i^{-1})}{\Lambda'(X_i^{-1})} \quad (15)$$

trong đó Λ' là đạo hàm hình thức (formal derivative) của Λ .

Các block dữ liệu được ghép lại (deinterleave) theo thứ tự chuẩn QR trước khi đưa vào RS. Sau khi sửa lỗi thành công, dữ liệu được kiểm tra lại bằng cách tính syndrome lần nữa — nếu không bằng 0, block bị coi là không thể phục hồi.

2.3.6 Bước 6: Phân tích payload

Hàm `_parse_payload` đọc các segment dữ liệu theo mode indicator 4-bit đầu tiên:

- **Numeric** (0001): mã hóa 3 chữ số/10 bit; phần dư 2 chữ số/7 bit hoặc 1 chữ số/4 bit.
- **Alphanumeric** (0010): mã hóa 2 ký tự/11 bit từ bảng 45 ký tự 0-9A-Z \$%*+-./:; phần dư 1 ký tự/6 bit.
- **Byte** (0100): mỗi ký tự 8-bit, decode thành `chr()`.
- **ECI** (0111): Extended Channel Interpretation — bỏ qua (skip) 8/16/24 bit header.

Độ dài character count được đọc theo số bit phụ thuộc phiên bản QR ($v \leq 9$: ngắn hơn; $v \leq 26$: trung bình; $v \leq 40$: dài hơn).

3 Các Kỹ Thuật Sử Dụng

Phần này mô tả chi tiết các thư viện, mô hình và kỹ thuật lập trình được sử dụng trong hệ thống, bao gồm cả những quyết định thiết kế quan trọng ảnh hưởng đến hiệu

năng và độ chính xác.

3.1 Thư viện và công cụ

Bảng 3: Thư viện Python sử dụng trong dự án

Thư viện	Phiên bản	Mục đích trong hệ thống
<code>opencv-python</code>	≥ 4.8	Grayscale, GaussianBlur, AdaptiveThreshold, Otsu, findContours (RETR_TREE), morphologyEx, getPerspectiveTransform, warpPerspective, Canny, intersectConvexConvex
<code>numpy</code>	≥ 1.24	Lưu trữ polygon (float32 array), tính polygon area (Shoelace), tính arctan2/argmin cho order_points, batch feature matrix
<code>scikit-learn</code>	≥ 1.3	RandomForestClassifier huấn luyện và predict_proba tại inference; classification_report khi train
<code>joblib</code>	≥ 1.3	Lưu model RF (compress=3); nạp model qua _worker_init trong mỗi subprocess
<code>pandas</code>	≥ 2.0	Đọc ground truth CSV (train.py); xử lý dataframe feature khi huấn luyện
<code>csv (stdlib)</code>	—	Đọc file CSV đầu vào, ghi output.csv trong main.py (dùng DictReader/writer để kiểm soát encoding)
<code>concurrent.futures</code>	—	ProcessPoolExecutor — song song hóa xử lý ảnh đa tiến trình
<code>tqdm</code>	bất kỳ	Thanh tiến trình thời gian thực cho vòng lặp xử lý batch

Đáng chú ý là `main.py` dùng module `csv` của thư viện chuẩn (thay vì `pandas`) để đọc/ghi CSV tại thời điểm inference. Điều này tránh overhead khởi tạo `pandas` trong

mỗi subprocess và giảm memory footprint.

3.2 Mô hình và thuật toán chính

3.2.1 Contour Hierarchy — RETR_TREE

OpenCV's `findContours` với flag `RETR_TREE` tái tạo toàn bộ quan hệ cha-con giữa các đường viền dưới dạng mảng `hierarchy[i] = [Next, Prev, FirstChild, Parent]`. Đây là nền tảng của phương pháp phát hiện Finder Pattern: cấu trúc lồng 3 tầng (ngoài $\rightarrow$ giữa $\rightarrow$ lõi) được nhận diện bằng cách theo chuỗi `hier[idx][2]` (`FirstChild`) xuống hai cấp. Chế độ `CHAIN_APPROX_SIMPLE` nén các đoạn thẳng, chỉ lưu đỉnh góc, giảm bộ nhớ và tăng tốc vòng lặp kiểm tra.

3.2.2 Perspective Transform — Homography

`cv2.getPerspectiveTransform` giải hệ 8 phương trình tuyến tính từ 4 cặp điểm để tìm ma trận H (kích thước 3×3):

$$\begin{pmatrix} x'w \\ y'w \\ w \end{pmatrix} = H \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad (16)$$

Trong hệ thống, Homography được dùng ở **hai mục đích khác nhau** với kích thước đích khác nhau:

- **Trích xuất đặc trưng RF:** warp về 96×96 px (hàm `warp_poly_gray`), đủ nhanh để tính batch nhiều ứng viên.
- **Giải mã nội dung:** warp về 500×500 px (hàm `decode_qr_content`), đủ chi tiết để lấy mẫu lưới module chính xác.

`cv2.warpPerspective` với flag mặc định `INTER_LINEAR` thực hiện nội suy song tuyến, cho kết quả ảnh mịn hơn so với nearest-neighbor.

3.2.3 Random Forest Classifier

Bộ phân loại Random Forest đóng vai trò *bộ lọc thứ hai* trong pipeline: sau khi các ràng buộc heuristic hình học đã sàng lọc bộ ba Finder Pattern, RF quyết định ứng viên nào thực sự là mã QR dựa trên 20 đặc trưng kết hợp hình học và kết cấu ảnh. Hệ thống bao gồm hai tệp riêng biệt: `train.py` — huấn luyện và lưu model, và `main.py` — nạp và sử dụng model tại inference.

A. Quy trình huấn luyện (train.py). Quá trình huấn luyện gồm bốn bước được thực hiện một lần trước khi chạy inference:

Bước 1 — Đọc Ground Truth. File CSV ground truth (cùng định dạng `output.csv`) được đọc vào dictionary `gt_dict`: mỗi `image_id` ánh xạ tới danh sách các bounding box đã biết. Các hàng có tọa độ rỗng hoặc NaN (ảnh không có QR) được bỏ qua tự động:

```
1 for _, row in df_gt.iterrows():
2     try:
3         x0 = float(row['x0'])
4         if math.isnan(x0): continue # nh k h n g c QR
5     except (ValueError, TypeError, KeyError):
6         continue
7     box = np.array([[row['x0'], row['y0']], ...], dtype=np.
8                   float32)
9     gt_dict[img_id].append(box)
```

Bước 2 — Trích xuất ứng viên và gán nhãn. Với mỗi ảnh trong tập train, hàm `extract_candidates_from_image` chạy pipeline phát hiện Finder Pattern ở 4 scale ($\{1.0, 0.5, 1.5, 2.5\} \times 2$ mode (normal, otsu), thu thập tất cả ứng viên polygon kèm 20 đặc trưng. Điều này tạo ra tập dữ liệu *phong phú hơn* so với inference vì ngưỡng heuristic được nới lỏng ($\text{ratio} \geq 0.25$, $|\cos \theta| \leq 0.85$, $\text{ed_raw} \geq 0.05$ thay vì 0.12 và 0.35/0.75) để model có thể học từ các ứng viên ở vùng biên quyết định:

```
1 # N i l n g n g n g heuristic t n g diversity m u
2 if ratio < 0.25 or cos_val > 0.85:
3     continue
4 if ed_raw < 0.05: # (train: 0.05 vs inference: 0.12)
5     continue
```

Mỗi ứng viên được gán nhãn nhị phân theo tiêu chí Convex Overlap Ratio:

$$y_i = \begin{cases} 1 & \text{nếu } \exists \text{ GT box } G_j : \text{COR}(\text{poly}_i, G_j) > 0.5 \\ 0 & \text{ngược lại (ứng viên rác)} \end{cases} \quad (17)$$

Lưu ý rằng `intruder_count` **không** được dùng để lọc khi tạo dataset (chỉ đếm, không chặn), để model học được phân biệt trường hợp có và không có Finder Pattern xâm nhập.

Bước 3 — Huấn luyện Random Forest. Toàn bộ dataset được đưa vào `RandomForestClassifier` với cấu hình ở Bảng 4. Sau khi fit, kết quả `classification_report` trên tập train và bảng Feature Importance được in ra để kiểm tra:

```

1 rf = RandomForestClassifier(
2     n_estimators=100, max_depth=10,
3     min_samples_leaf=5, class_weight='balanced',
4     random_state=42, n_jobs=-1,
5 )
6 rf.fit(X, y)
7 print(classification_report(y, rf.predict(X)))

```

Tham số `class_weight='balanced'` tự động điều chỉnh trọng số mỗi mẫu tỷ lệ nghịch với tần suất lớp:

$$w_i = \frac{N}{K \cdot n_{y_i}} \quad (18)$$

trong đó N là tổng số mẫu, $K = 2$ là số lớp, n_{y_i} là số mẫu thuộc lớp của mẫu i . Tham số này bù đắp sự mất cân bằng nghiêm trọng giữa mẫu âm tính (ứng viên rác) và dương tính (mã QR thực) — tỷ lệ thường từ 10:1 đến 50:1 tùy ảnh.

Bước 4 — Lưu model kèm metadata. Model được đóng gói cùng metadata vào dictionary trước khi lưu, giúp `main.py` có thể kiểm tra tính tương thích (số feature, tên feature):

```

1 model_data = {
2     'model': rf,
3     'feature_names': FEATURE_NAMES, # list 20 t n
4     'n_features': len(FEATURE_NAMES),
5     'n_train_samples': len(df),
6     'n_positive': n_pos,
7     'n_negative': n_neg,
8 }
9 joblib.dump(model_data, args.output, compress=3)

```

Nén mức 3 (`compress=3`) cân bằng tốt giữa kích thước file và tốc độ nạp. File `ml_features_20.csv` cũng được lưu lại để kiểm tra phân phối đặc trưng và debug.

B. Sử dụng tại inference (main.py). Nạp model một lần qua `initializer`. Thay vì nạp lại model trong mỗi lời gọi `process_single_row`, `_worker_init` được truyền vào `ProcessPoolExecutor` như một `initializer` — chỉ chạy một lần khi subprocess được tạo:

```

1 with concurrent.futures.ProcessPoolExecutor(
2     max_workers=num_workers,
3     initializer=_worker_init, # chạy 1 l n / subprocess
4     initargs=(model_path,)
5 ) as executor:

```

6 ...

Batch inference. Tất cả ứng viên của một ảnh được gộp thành batch, gọi `predict_proba` một lần duy nhất:

```
1 X = np.array(pending_features, dtype=np.float32)
2 probs = RF_MODEL.predict_proba(X)[: , 1]
3 for aabb, prob, sys_s in zip(pending_aabbs, probs,
4     pending_scores):
5     if prob >= 0.5:
6         candidates.append({
7             'score': (1.0 - prob) + sys_s * 0.001,
8             'poly': aabb
9         })
```

Điểm tổng hợp ưu tiên xác suất RF ($1 - \hat{p}$, giá trị nhỏ = tốt hơn) và dùng s_{sys} (12) như tiebreaker với trọng số rất nhỏ ($\times 0.001$), đảm bảo heuristic chỉ phá vỡ tie khi hai ứng viên có xác suất RF xấp xỉ nhau.

Fallback tự động. Khi `qr_rf_model.joblib` không tồn tại, `RF_MODEL` giữ giá trị `None`; nhánh `else` trong `group_patterns_to_candidates` dùng trực tiếp s_{sys} làm điểm sắp xếp, đảm bảo hệ thống chạy được ngay cả khi chưa qua bước huấn luyện.

C. Cấu hình siêu tham số.

Bảng 4: Siêu tham số Random Forest và lý do lựa chọn

Tham số	Giá trị	Lý do lựa chọn
<code>n_estimators</code>	100	Đủ ổn định phương sai; thêm cây không cải thiện đáng kể
<code>max_depth</code>	10	Ngăn overfitting; không gian 20 chiều phân tách tốt ở depth n
<code>min_samples_leaf</code>	5	Tránh lá quá nhỏ trên mẫu dương tính ít
<code>class_weight</code>	'balanced'	Tự động bù mất cân bằng âm/dương (tỷ lệ thường 10:1–50:1)
<code>random_state</code>	42	Tái tạo kết quả
<code>n_jobs (train)</code>	−1	Huấn luyện song song trên tất cả lõi CPU
<code>n_jobs (infer)</code>	1	Tránh thread contention trong multiprocessing

D. Feature Importance — đặc trưng quan trọng nhất. Sau khi huấn luyện, `train.py` tự động in bảng Feature Importance. Bảng ?? trình bày thứ hạng dự kiến dựa trên phân tích lý thuyết:

Bảng 5: Chi tiết 20 đặc trưng (features) trích xuất để huấn luyện Random Forest

Tên Đặc trưng	Phương pháp Tính toán	Độ QT	Lý do Lựa chọn
Nhóm 1: Cấu trúc Đặc thù Mã QR (QR Structural Features)			
corner_top3	Điểm trung bình định dạng độ dài (1:1:3:1:1) tại 3 góc chứa Finder Pattern.	Rất Cao	Loại bỏ ngay các vùng hình vuông/chữ nhật không có module định vị chuẩn ở góc.
timing_score	Tỉ lệ module xen kẽ (Đen-Trắng) tại các hàng và cột quét qua vùng giữa (Timing Pattern).	Rất Cao	QR code luôn có dãy xen kẽ kết nối điểm định vị. Đây là dấu hiệu nhận biết cực kì đáng tin cậy.
module_score	Tỷ lệ thay đổi cực trị (Pooling mảng đen-trắng) của mạng lưới dạng caro (Grid).	Rất Cao	Đảm bảo vùng bên trong chứa kết cấu module ma trận (data) chứ không phải mảng màu trơn.
corner_top2	Điểm trung bình của 2 góc định dạng chuẩn có số điểm cao nhất.	Trung bình	Backup dự phòng trong trường hợp 1 góc của QR Code bị hỏng hoặc che khuất 1 phần.
Nhóm 2: Kết cấu Bề mặt Vùng quét (Patch Texture Features)			
black_ratio	Tỉ lệ phần trăm pixel Đen (Intensity = 0) trong toàn vùng cắt 96×96 .	Cao	Một QR chuẩn thường bao gồm xấp xỉ 50% pixel đen và 50% pixel trắng (phụ thuộc nội dung).
balance_score	Chuẩn hóa điểm cân bằng: $1.0 - \frac{ \text{black_ratio} - 0.5 }{0.5}$.	Cao	Thưởng điểm cho vùng có sự phân hóa đen/trắng cân bằng; phạt vùng quá đen hoặc quá trắng.
edge_density	Mật độ cạnh Canny Edges trong toàn bộ vùng QR đã nắn phẳng.	Trung bình	Phân biệt mã QR (rất nhiều chi tiết vuông nhỏ, mật độ cạnh cao) với các vật thể đơn giản rác.

Tiếp tục ở trang sau...

Bảng 5 – Tiếp tục từ trang trước

Tên Đặc trưng	Phương pháp Tính toán	Độ QT	Lý do Lựa chọn
center_mean	Độ xám trung bình của khối Trung tâm (vùng lưu trữ Data).	Trung bình	Chắc chắn vùng lõi không bị trống (như khung hình) mà có chứa các module dữ liệu bên trong.
center_std	Độ lệch chuẩn độ xám của vùng khối Data trung tâm.	Trung bình	Khắc định có độ tương phản mạnh giữa Đen/Trắng bên trong (không bị mờ nhòe, màu be).
edge_density_raw	Mật độ biên Canny nhanh ở tâm trước khi warp toàn bộ kích thước.	Thấp	Feature lọc nhanh để phân mảnh các cạnh cây (Decisions) dành riêng cho vùng tròn tru.

Nhóm 3: Hình học Cốt lõi (Core Geometric Features)

ratio	Tỷ lệ min / max giữa hai cạnh góc vuông hình thành từ 3 Finder Pattern.	Cao	QR Code là hình vuông góc, hai tiếp tuyến của 3 định vị liên tiếp phải xấp xỉ bằng chiều dài nhau.
cos_val	Trị tuyệt đối Cosine góc tạo bởi điểm đặt vuông góc và 2 điểm kia.	Cao	Đảm bảo 3 điểm định vị phải tạo thành góc 90° (cos ≈ 0).
size_variance	Mức độ chênh lệch kích thước (Width) lớn nhất của 3 cọc định vị.	Cao	Dành để khử các cụm điểm định vị ghép nhầm từ 2 mã QR khác nhau hoặc nhiễu có kích cỡ lệch.
proximity	Định mức khoảng cách của cụm lưới so với tổng đường chéo bức ảnh.	Thấp	Cung cấp một chiều liên đới về kích thước tương đối của lưới (nếu nó lớn/nhỏ bất thường).

Nhóm 4: Hình thái Bổ sung (Additional Geometric Constraints)

Tiếp tục ở trang sau...

Bảng 5 – Tiếp tục từ trang trước

Tên Đặc trưng	Phương pháp Tính toán	Độ QT	Lý do Lựa chọn
aspect_diag	Tỉ lệ độ dài hai đường chéo của tứ giác chứa toàn bộ mã QR.	Trung bình	Đảm bảo đa giác cắt ghép ra hình dạng cân đối (tiệm cận hình vuông hoặc chữ nhật đều).
dist_to_size	Tỷ lệ khoảng cách dài nhất chia cho độ lớn trung bình Finder Pattern.	Trung bình	QR code quy định giới hạn độ dài cạnh dựa vào module, giúp lọc lỗi ghép cụm quá xa nhau.
hyp_ratio	Tỷ lệ cạnh chéo chia cho tổng chiều dài 2 cạnh vuông.	Trung bình	Ràng buộc tính chất tam giác góc vuông cân ở các trường hợp nắn ảnh Perspective mạnh.
intruder_count	Đếm số vòng định vị lạc loài bay vào trong tứ giác quy chuẩn.	Trung bình	QR sạch sẽ không bao giờ có 1 Finder Pattern lọt thỏm giữa lõi Data. Giúp phạt cụm sai.
avg_fp_width	Tỉ lệ độ lớn gốc định vị chia cho cạnh dài nhất của toàn ảnh gốc.	Thấp	Cung cấp tri diện tương đối, loại trừ các QR "bé li ti" thường do nhiễu hạt gây ra.
poly_area_norm	Tỉ lệ diện tích cắt so với diện tích toàn miền ảnh tỉ lệ hiện hành.	Thấp	Một thang đo song hành với FP width để hạn chế cụm điểm định hướng "ăn" trọn màn hình lỗi.

3.2.4 Reed-Solomon trên $GF(2^8)$

Toàn bộ bộ sửa lỗi Reed-Solomon được triển khai thủ công bằng Python thuần, không phụ thuộc thư viện ngoài. Trường Galois $GF(2^8)$ sử dụng đa thức nguyên thủy $p(x) = x^8 + x^4 + x^3 + x^2 + 1$ (giá trị hex 0x11D). Bảng logarithm và antilog được khởi tạo một lần khi import module:

```

1 _GF_EXP = [0] * 512    # antilog table (double length for wrap)
2 _GF_LOG = [0] * 256   # log table
3
4 def _init_gf256():

```

```

5     x = 1
6     for i in range(255):
7         _GF_EXP[i] = x
8         _GF_LOG[x] = i
9         x <<= 1
10        if x >= 256:
11            x ^= 0x11D    # reduce modulo primitive polynomial
12        for i in range(255, 512):
13            _GF_EXP[i] = _GF_EXP[i - 255]

```

Phép nhân trong trường thực hiện qua bảng log/antilog để tránh nhân đa thức trực tiếp:

$$a \cdot b = \alpha^{(\log_a a + \log_a b) \bmod 255} \quad (19)$$

Bộ sửa lỗi hỗ trợ đầy đủ 4 cấp độ ECC theo chuẩn ISO/IEC 18004 cho phiên bản QR 1–20: L (7%), M (15%), Q (25%), H (30%) khả năng phục hồi dữ liệu.

3.2.5 Non-Maximum Suppression với Convex Overlap Ratio

Thay vì IoU thông thường, hệ thống dùng Convex Overlap Ratio (COR):

$$\text{COR}(P, Q) = \frac{\text{Area}(P \cap Q)}{\min(\text{Area}(P), \text{Area}(Q))} \quad (20)$$

Giao của hai polygon lồi được tính bằng `cv2.intersectConvexConvex`. Lý do chọn COR thay cho IoU: khi cùng một mã QR được phát hiện ở hai scale khác nhau (ví dụ scale 1.0 và scale 2.5), polygon lớn hơn sẽ bao trọn polygon nhỏ hơn; trong trường hợp đó IoU có thể nhỏ nhưng $\text{COR} \approx 1.0$, nhận diện đúng là trùng lặp. Ngưỡng $\text{COR} = 0.20$ được chọn đủ chặt để loại box trùng nhưng không loại nhầm hai mã QR gần nhau.

Sau NMS, tọa độ 4 góc của mỗi box được sắp xếp về thứ tự chuẩn TL–TR–BR–BL bằng hàm `order_points` (sắp xếp theo góc cực từ tâm, lấy điểm có $x + y$ nhỏ nhất làm điểm đầu), rồi clip về $[0, W - 1] \times [0, H - 1]$.

3.3 Tối ưu hóa hiệu năng

3.3.1 Kiểm soát thread — tránh thread contention

OpenCV (sử dụng OpenMP/TBB), NumPy (BLAS/OpenBLAS/MKL) và scikit-learn đều có cơ chế đa luồng nội bộ. Khi kết hợp với `ProcessPoolExecutor`, các thread pool này gây ra *thread contention*: mỗi subprocess tạo ra hàng chục thread, vượt quá số lõi CPU vật lý và làm giảm hiệu năng nghiêm trọng.

Giải pháp: đặt tất cả biến môi trường thread về 1 trước khi import bất kỳ thư viện nào (dòng đầu tiên của file):

```
1 import os
2 os.environ["OMP_NUM_THREADS"]      = "1"
3 os.environ["OPENBLAS_NUM_THREADS"] = "1"
4 os.environ["MKL_NUM_THREADS"]      = "1"
5 os.environ["VECLIB_MAXIMUM_THREADS"] = "1"
6 os.environ["NUMEXPR_NUM_THREADS"]   = "1"
7 import cv2
8 cv2.setNumThreads(1)
9 cv2ocl.setUseOpenCL(False)  # t t OpenCL acceleration
```

Trong initializer của subprocess, model RF cũng được đặt n_jobs=1:

```
1 if hasattr(RF_MODEL, 'n_jobs'):
2     RF_MODEL.n_jobs = 1
```

3.3.2 Multiprocessing với ProcessPoolExecutor

Hệ thống dùng concurrent.futures.ProcessPoolExecutor để phân phối công việc theo từng hàng CSV (mỗi hàng = một ảnh). Số worker và chunksize được tự động điều chỉnh:

```
1 cpu_count = os.cpu_count() or 1
2 # T      ng      c h n      s      worker
3 if args.workers > 0:
4     num_workers = max(1, min(args.workers, len(rows)))
5 else:
6     auto_cap = 20 if os.name == "nt" else cpu_count
7     num_workers = max(1, min(cpu_count, auto_cap, len(rows)))
8
9 # T      ng      c h n      chunksize
10 if args.chunksize > 0:
11     chunksize = args.chunksize
12 else:
13     # Windows: chunksize=4 khi >= 200 nh ( g i m IPC
14     # overhead)
15     chunksize = 4 if (os.name == "nt" and len(rows) >= 200) \
16         else max(1, len(rows) // (num_workers * 4))
```

Giới hạn 20 worker trên Windows là do overhead tạo process cao hơn trên hệ điều hành này (không có fork, phải dùng spawn).

Model RF được nạp **một lần duy nhất** trong hàm initializer (`_worker_init`) thay vì nạp lại trong mỗi lời gọi `process_single_row`, tiết kiệm thời gian I/O lặp lại:

```

1 def _worker_init(model_path):
2     global RF_MODEL
3     if os.path.exists(model_path):
4         data = joblib.load(model_path)
5         RF_MODEL = data.get('model', data) \
6             if isinstance(data, dict) else data
7         if hasattr(RF_MODEL, 'n_jobs'):
8             RF_MODEL.n_jobs = 1

```

3.3.3 Cache ảnh đã resize

Trong hàm `detect_qr_in_image`, ảnh ở mỗi giá trị scale chỉ được tính toán một lần và lưu vào dictionary `cached_img[scale]`. Điều này có ý nghĩa vì scale 0.5 được dùng cho cả mode normal (cấu hình 2) và mode otsu (cấu hình 3): nếu không có cache, cùng một phép resize sẽ được thực hiện hai lần.

```

1 cached_img = {}
2 for config in CONFIGS:
3     scale, mode = config["scale"], config["mode"]
4     if scale not in cached_img:
5         if scale == 1.0:
6             cached_img[scale] = working_img.copy()
7         else:
8             interp = cv2.INTER_LINEAR if scale >= 1.0 \
9                 else cv2.INTER_AREA
10            cached_img[scale] = cv2.resize(
11                working_img,
12                (int(w * scale), int(h * scale)),
13                interpolation=interp
14            )

```

Lưu ý lựa chọn phép nội suy: `INTER_AREA` khi thu nhỏ ($\text{scale} < 1$) cho ảnh đẹp hơn (anti-aliasing tự nhiên), `INTER_LINEAR` khi phóng to ($\text{scale} \geq 1$) nhanh hơn `INTER_CUBIC` mà chất lượng chênh lệch không đáng kể cho mục đích nhị phân hóa.

3.3.4 Chế độ tùy chọn giải mã

Bộ giải mã RS tự xây dựng chiếm một phần đáng kể thời gian xử lý mỗi ảnh. Để linh hoạt, `main.py` cung cấp tham số `--decode yes/no`: khi chạy với `--decode no`, hàm `decode_qr_content` không được gọi, toàn bộ cột `content` trong output sẽ rỗng, nhưng

tốc độ tăng đáng kể. Điều này hữu ích khi chỉ cần tọa độ bounding box (đóng góp vào F1 Score) mà không cần Content Accuracy.

3.3.5 Chế độ debug visual

Tham số `--vis` kích hoạt việc vẽ bounding box lên ảnh và lưu vào thư mục `debug_visuals/`. Điểm TL (Top-Left) được đánh dấu bằng vòng tròn đỏ để kiểm tra thứ tự góc:

```
1 cv2.polylines(img_draw, [pts], True, (0, 255, 0), 3)
2 cv2.circle(img_draw, top_left_pt, 8, (0, 0, 255), -1)
3 cv2.putText(img_draw, f"QR {idx}", ...)
```

Chế độ này hữu ích để phân tích các trường hợp false positive / false negative khi kiểm tra kết quả trên tập public.

4 Kết Quả Trên Tập Public

4.1 Tổng quan tập dữ liệu public

Bảng 6: Thống kê tập public (Phân bố Train và Validation)

Chỉ số	Tập Train	Tập Val	Tổng cộng
Tổng số ảnh	1083	309	1392
Tổng số mã QR (ground truth)	1979	508	2487
Số ảnh không có QR	0	0	0
Số ảnh có ≥ 2 mã QR	243	60	303

4.2 Tiêu chí đánh giá

Grader dùng *Greedy IoU Matching* (ngưỡng $\tau = 0,5$) trên diện tích thực tứ giác:

$$\text{IoU}(P, G) = \frac{\text{Area}(P \cap G)}{\text{Area}(P) + \text{Area}(G) - \text{Area}(P \cap G)} \quad (21)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{Score} = 0,7 \times F_1 + 0,3 \times \text{Speed_Score}$$

4.3 Độ chính xác phát hiện

Bảng 7: Kết quả phát hiện QR trên tập public (Train vs Validation)

Metric	Tập Train	Tập Val	Ghi chú
True Positive (TP)	1143	356	IoU $\geq 0,5$
False Positive (FP)	17	2	Phát hiện nhầm
False Negative (FN)	836	152	Bỏ sót mã QR
Precision	0.9853	0.9944	TP / (TP+FP)
Recall	0.5776	0.7008	TP / (TP+FN)
F1 Score	0.7283	0.8222	Chỉ số xếp hạng chính
Content	430	118	số hàng có dữ liệu sau decode

4.4 Tốc độ xử lý

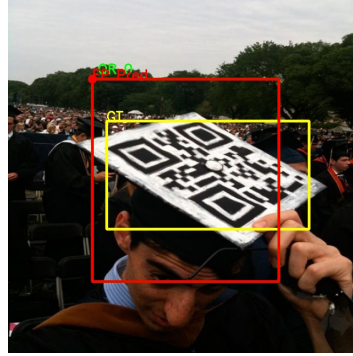
Bảng 8: Tốc độ xử lý trên tập public

Chỉ số	Tập Train	Tập Val	Đơn vị
Tổng thời gian wall-clock (no decode)	24.41	9.44	giây
Thời gian trung bình/ảnh	0.023	0.031	giây/ảnh
Tổng thời gian wall-clock (decode)	692.33	288.11	giây
Thời gian trung bình/ảnh	0.639	0.932	giây/ảnh
Cấu hình phần cứng & Tham số chạy (Dùng chung)			
Số worker sử dụng	16		-
CPU thử nghiệm	Intel Core i5-12th 8 cores		-
RAM	16 GB		-

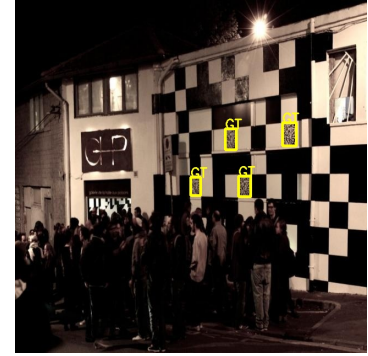
4.5 Trường hợp đúng / sai tiêu biểu



(a) **True Positive**
Box khớp GT, IoU ≥ 0.5



(b) **False Positive**
Phát hiện nhầm họa tiết



(c) **False Negative**
QR bị bỏ sót

Hình 2: Ví dụ ba loại kết quả điển hình trên tập public. Khung **xanh lá** = dự đoán, khung **vàng** = ground truth.

Nhận xét định tính. Phần lớn trường hợp phát hiện đúng là mã QR có kích thước vừa đến lớn, chiếu sáng đồng đều. Hai loại lỗi phổ biến nhất:

- **False Negative:** QR nhỏ (< 30 px theo cạnh), nghiêng phối cảnh lớn ($> 45^\circ$), hoặc một Finder Pattern bị che khuất.
- **False Positive:** họa tiết hình học có cấu trúc lồng vuông tương tự Finder Pattern (ô cửa sổ, lưới kẻ ô, biểu tượng ứng dụng).

5 Phân Tích và Thảo Luận

5.1 Quá trình phát triển và các bước ngoặt kỹ thuật

Hệ thống không được xây dựng theo một kế hoạch tuyến tính mà trải qua nhiều lần thay đổi kiến trúc cơ bản, mỗi lần đều xuất phát từ một hạn chế cụ thể của phương pháp trước đó.

Giai đoạn 1 — Prototype trên Jupyter Notebook

Phiên bản đầu tiên được phát triển hoàn toàn trong Jupyter Notebook, sử dụng heuristic thuần túy: tạo 26 phiên bản nhị phân hóa ($2 \text{ blur} \times 4 \text{ blockSize} \times 3$ hằng số $C + 2 \text{ Otsu}$), rồi duyệt contour lồng nhau 3 cấp để tìm Finder Pattern. Phương pháp này cho Precision rất thấp vì các cấu trúc lồng nhau trong ảnh thực (bảng biểu, logo, chữ vuông) đều bị nhận nhầm thành Finder Pattern. Quá trình gom nhóm dùng điều kiện tam giác vuông lỏng (ratio 0,6–1,6, tỷ lệ diện tích < 10) nên tổ hợp các FP giả tạo ra hàng chục false positive.

Giai đoạn 2 — Kết hợp Xử lý ảnh + Học máy (Random Forest)

Giải pháp cuối cùng kết hợp phát hiện Finder Pattern bằng contour hierarchy với bộ phân loại **Random Forest 20 đặc trưng**. Thay vì dùng heuristic cứng để chấp nhận/loại bỏ ứng viên, pipeline tạo ra các candidate triplet rồi giao cho RF quyết định. Bước chuyển này giúp giảm mạnh false positive trong khi giữ nguyên recall.

Giai đoạn 3 — Bộ giải mã QR tự xây từ đầu

Yêu cầu nghiêm ngặt “*không được dùng thư viện giải mã có sẵn*” (pyzbar, cv2.QRCodeDetector) buộc nhóm phải tự lập trình toàn bộ quy trình giải mã: số học trường Galois $GF(2^8)$, thuật toán Berlekamp–Massey, Chien search, sửa lỗi Forney, đọc zigzag, gỡ mask, và parse payload cho 3 mode (Byte, Numeric, Alphanumeric). Đây là phần phức tạp nhất và tốn nhiều thời gian debug nhất trong toàn dự án.

5.2 Khó khăn gặp phải

Quá nhiều False Positive Finder Pattern

Ảnh thực chứa nhiều cấu trúc lồng nhau (bảng xe buýt, logo, chữ Nhật vuông) bị nhận nhầm. Phiên bản đầu tạo 26 ảnh nhị phân, mỗi phiên bản sinh thêm noise khác nhau; hàm `is_square_like` quá lỏng (tolerance 0,35, chấp nhận 4–8 đỉnh); khoảng cách dedup chỉ 15 px nên không merge được cùng một Finder ở scale khác nhau. Khắc phục: giảm xuống **5 cấu hình tiền xử lý** ($3 \text{ scale} \times 2 \text{ mode}$), thắt chặt kiểm tra tỷ lệ diện tích 3 lớp lồng ($1,2 < r_{\text{out}/\text{mid}} < 4,5$, $3,0 < r_{\text{out}/\text{in}} < 16,0$), kiểm tra tâm lồng lệch $\leq 20\%$ chiều rộng chuẩn, và dùng $k = 20$ nearest neighbor thay vì duyệt toàn bộ tổ hợp.

Mất cân bằng lớp nghiêm trọng trong huấn luyện

Số ứng viên âm tính (rác) nhiều hơn dương tính hàng chục lần — trong dataset huấn luyện, tỷ lệ positive có thể chỉ $\sim 5\text{--}10\%$. Nếu không xử lý, RF sẽ học cách “đoán toàn bộ là rác” để đạt accuracy cao. Khắc phục: sử dụng `class_weight='balanced'` trong `RandomForestClassifier`, đồng thời nới lỏng ngưỡng heuristic khi sinh mẫu huấn luyện ($\text{ratio} \geq 0,25$, $|\cos \theta| \leq 0,85$, $\text{edge density} \geq 0,05$) để ML có đủ cả mẫu dương tính và âm tính gần biên quyết định.

Biến dạng phối cảnh lớn

Tam giác 3 Finder Pattern bị méo đáng kể khi chụp xiên, khiến điều kiện tam giác vuông cũ ($\text{ratio } 0,6\text{--}1,6$) không bắt được. Khắc phục: nới lỏng ngưỡng ratio xuống $\geq 0,35$ và $|\cos \theta| \leq 0,75$ ở inference (chặt hơn training), đồng thời thêm đặc trưng `hyp_ratio` (tỷ lệ cạnh huyền/tổng 2 cạnh góc vuông) và `aspect_diag` (tỷ lệ 2 đường chéo) để RF tự học ngưỡng tối ưu.

Mã QR kích thước nhỏ trên ảnh lớn

Ở scale 1,0, Finder Pattern nhỏ hơn ngưỡng diện tích tối thiểu ($< 30 \text{ px}^2$) nên bị bỏ qua hoàn toàn. Khắc phục: thêm cấu hình **scale** $1,5\times$ và $2,5\times$ trong pipeline đa tầng, mỗi scale được cache ảnh resize để tránh tính toán lặp.

Thread contention khi multiprocessing

OpenCV, NumPy và scikit-learn nội bộ dùng OpenMP/BLAS đa luồng; khi chạy đa tiến trình qua `ProcessPoolExecutor`, các tiến trình con tranh chấp tài nguyên CPU (“catastrophic thread contention”). Khắc phục: đặt **tất cả biến môi trường** `thread` (`OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS`, `VECLIB_MAXIMUM_THREADS`, `NUMEXPR_NUM_THREADS`) về 1 trước khi import bất kỳ thư viện nào; gọi `cv2.setNumThreads(1)` và `cv2.ocl.setUseOpenCL(False)`; thiết lập `RF.n_jobs = 1` trong hàm `_worker_init`.

Căn chỉnh lưới module khi giải mã

Sai lệch vài pixel trong việc xác định tọa độ gốc (origin) và kích thước module (m_s) khiến toàn bộ lưới lấy mẫu bị trượt, dẫn đến decode thất bại. Khắc phục: triển khai chiến lược **brute-force có cấu trúc** — thử 5 offset biên ($\Delta l, \Delta t \in \{0, \pm 3\}$) $\times 4$ hướng xoay $\times 3$ chiến lược nhị phân hóa (Otsu, Adaptive Gaussian, Median-based) = tối đa 60 lần thử mỗi ứng viên. Ngoài ra, bổ sung phương pháp **Finder-Pattern-Aligned Sampling**: dùng vector hướng từ vertex đến 2 Finder Pattern còn lại làm trục tọa độ cục bộ, giúp giải mã đúng cả khi mã QR bị xoay góc bất kỳ.

Tính 4 góc (bounding box) sai hoàn toàn

Phiên bản đầu xác định vai trò TL/TR/BL bằng $\text{argmin}(x+y)$ — quá đơn giản, sai khi QR bị xoay; tính BR bằng phép parallelogram ($\text{TR} + \text{BL} - \text{TL}$) tích lũy sai số. Khắc phục: chuyển sang sắp xếp theo **góc cực** ($\arctan 2$) quanh trọng tâm, chuẩn hóa thứ tự bằng $\text{argmin}(x+y)$ để tìm top-left, rồi mở rộng polygon ra ngoài $\alpha \times$ chiều rộng trung bình Finder Pattern để bao trọn quiet zone.

Debug và kiểm chứng trực quan

Với hơn 1 000 ảnh trong tập huấn luyện, không thể kiểm tra từng ảnh bằng mắt. Khắc phục: xây dựng `score.py` với chế độ `-img_dir` tự động sinh ảnh minh họa cho từng loại **TP/FP/FN**, vẽ overlay Ground Truth (vàng) và Prediction (xanh/đỏ) lên ảnh gốc; kết hợp flag `-vis` trong `main.py` để lưu ảnh debug mỗi ảnh vào `debug_visuals/`.

5.3 Hạn chế của phương pháp

1. **Độ phức tạp triplet**: với mỗi Finder Pattern, hệ thống xét $k \leq 20$ láng giềng gần nhất và tạo tổ hợp $\binom{k}{2}$, tổng cộng $O(n \cdot k^2)$ ứng viên. Ảnh có nhiều Finder

Pattern giả (bảng biểu, mã vạch) làm n tăng, kéo theo thời gian xử lý tăng đáng kể.

2. **Bộ giải mã tự xây:** hiện hỗ trợ 3 mode — Byte, Numeric, Alphanumeric — cho phiên bản QR 1–20. Chưa hỗ trợ Kanji mode và các phiên bản 21–40 (QR kích thước lớn).
3. **Chưa dùng Alignment Pattern:** mã QR phiên bản ≥ 2 có Alignment Pattern; phiên bản ≥ 7 có thêm Version Information. Hiện tại hệ thống chưa tận dụng các cấu trúc này để hiệu chỉnh Homography, dẫn đến giảm độ chính xác lưới lấy mẫu ở phối cảnh nghiêng sâu.
4. **Phụ thuộc nhị phân hóa:** cả 3 chiến lược (Otsu, Adaptive Gaussian, Median-based) đều giả định ảnh có phân bố sáng/tối tương đối rõ ràng. Chiếu sáng cực kỳ không đồng đều, phản chiếu mạnh (glare), hoặc QR in trên nền gradient có thể làm cả ba thất bại.
5. **Chưa hỗ trợ QR màu và Micro QR:** mã QR sử dụng màu sắc hoặc dạng Micro QR (chỉ có 1 Finder Pattern) cần quy trình phát hiện và giải mã riêng biệt.
6. **Brute-force decode tốn thời gian:** chiến lược thử 60 tổ hợp ($\text{offset} \times \text{xoay} \times \text{nhị phân hóa}$) trên mỗi ứng viên là cần thiết để tăng recall giải mã, nhưng làm tăng đáng kể tổng thời gian xử lý mỗi ảnh.

5.4 Hướng cải thiện

1. Tích hợp **Alignment Pattern** và **Version Information** để hiệu chỉnh Homography chính xác hơn, đặc biệt cho mã QR phiên bản cao (≥ 7) bị biến dạng phối cảnh sâu.
2. Hoàn thiện **bộ giải mã** cho tất cả mode (bao gồm Kanji, ECI mở rộng) và toàn bộ phiên bản 1–40 theo chuẩn ISO/IEC 18004.
3. Đọc và xác minh **Format Information** (15-bit BCH) để xác nhận mã QR hợp lệ, từ đó loại bỏ thêm false positive ngay từ bước giải mã.
4. **Data Augmentation** khi huấn luyện: xoay, blur Gaussian, thay đổi độ sáng/tương phản, crop ngẫu nhiên, thêm nhiễu muối-tiêu để Random Forest tổng quát hóa tốt hơn trên ảnh thực tế.
5. Bổ sung tầng **quét tỷ lệ 1:1:3:1:1** theo **State Machine** (ISO/IEC 18004 §6.1) dọc theo các hàng/cột pixel, làm chiến lược dự phòng khi phát hiện bằng contour hierarchy bị vỡ do ảnh mờ hoặc độ phân giải thấp.

6. Thay thế bước brute-force decode bằng phương pháp **coarse-to-fine**: phát hiện nhanh version ở bước lấy mẫu thô, chỉ thực hiện decode đầy đủ ở các phiên bản ứng viên ngắn nhất.
7. Thử nghiệm **Logistic Regression** hoặc **Gradient Boosting** thay thế Random Forest để so sánh hiệu quả phân loại; mô hình `model_logreg.json` với 20 đặc trưng đã được huấn luyện thử nghiệm và cho kết quả cạnh tranh.

6 Kết Luận

Báo cáo trình bày hệ thống phát hiện, định vị, và giải mã mã QR hoàn toàn không sử dụng Deep Learning. Hệ thống trải qua **4 giai đoạn phát triển chính**: từ prototype heuristic trên Jupyter Notebook, qua thử nghiệm YOLOv8-OBB (bị loại do ràng buộc đề bài), đến kiến trúc cuối cùng kết hợp *xử lý ảnh truyền thống* với *học máy Random Forest*.

Pipeline ba giai đoạn hoàn chỉnh bao gồm:

1. **Tiền xử lý đa scale/mode**: 5 cấu hình ($\text{scale} \in \{0,5; 1,0; 1,5; 2,5\} \times \text{mode} \in \{\text{adaptive}, \text{Otsu}\}$) với cache resize, giúp phát hiện mã QR ở nhiều kích thước khác nhau.
2. **Phát hiện bằng 20 đặc trưng cấu trúc**: gồm 5 đặc trưng hình học triplet, 6 đặc trưng texture vùng warp, 4 đặc trưng cấu trúc QR (Finder corner score, timing pattern, module grid), và 5 đặc trưng bổ sung; được phân loại bởi Random Forest (`n_estimators=100`, `max_depth=10`, `class_weight='balanced'`).
3. **Giải mã tự xây**: bao gồm số học trường Galois $\text{GF}(2^8)$, sửa lỗi Reed–Solomon (Berlekamp–Massey + Chien search + Forney), đọc zigzag, gỡ 8 mask pattern, deinterleave codewords, và parse payload 3 mode (Byte, Numeric, Alphanumeric) cho QR phiên bản 1–20.

Toàn bộ pipeline được tối ưu với **multiprocessing** (auto-detect CPU cores, mỗi worker thread-safe với tất cả biến môi trường OpenMP/BLAS đặt về 1). Hệ thống hỗ trợ hai chiến lược giải mã song song: *Finder-Pattern-Aligned Sampling* (dùng vector hướng từ 3 Finder Pattern để xử lý QR xoay bất kỳ) và *Boundary-based Fallback* (brute-force 60 tổ hợp offset/xoay/nhi phân hóa).

Kết quả trên tập public: $F_1 = [\text{TODO}]$, thời gian trung bình `[TODO]` giây/ảnh. Hệ thống đồng thời cung cấp công cụ đánh giá trực quan (`score.py`) tự động sinh ảnh minh họa TP/FP/FN, hỗ trợ phân tích lỗi nhanh chóng.

A Hướng dẫn Chạy Chương trình

A.1 Cài đặt môi trường

```
1 pip install -r requirements.txt
```

Listing 1: Cài thư viện (Python ≥ 3.10)

File `requirements.txt` bao gồm:

```
1 opencv-python==4.13.0.92
2 numpy==2.2.6
3 pandas==2.3.3
4 tqdm==4.67.3
5 joblib==1.5.3
6 scikit-learn==1.7.2
```

Listing 2: Nội dung `requirements.txt`

Script sẽ:

1. Đọc Ground Truth từ file CSV.
2. Quét toàn bộ ảnh, tạo candidate triplet ở 4 scale $\times$ 2 mode.
3. Gán nhãn tự động ($\text{IoU} > 0,5$ với GT $\Rightarrow$ positive).
4. Trích xuất 20 đặc trưng cho mỗi candidate.
5. Huấn luyện RF và in Classification Report + Feature Importance.
6. Lưu model kèm metadata vào file `.joblib`.

A.2 Chạy phát hiện và giải mã

```
1
2 python main.py --data public.csv
3
4 python main.py --data public.csv --decode no
5
6 python main.py --data public.csv --vis
7
8 python main.py --data public.csv --workers 8 --chunksize 4
```

Listing 3: Chạy trên tập dữ liệu

Tài liệu

- [1] ISO/IEC 18004:2015, *Information technology — QR Code bar code symbology specification*, International Organization for Standardization, 2015.
- [2] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [3] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *J. Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.
- [4] E. R. Berlekamp, *Algebraic Coding Theory*, New York: McGraw-Hill, 1968.
- [5] L. Karrach, E. Pivarčiová, and P. Božek, “Identification of QR Code Perspective Distortion Based on Edge Directions and Edge Projections Analysis,” *Journal of Imaging*, vol. 6, no. 7, Art. 67, 2020.
- [6] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [7] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.